МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ТАРАСА ШЕВЧЕНКА
ФАКУЛЬТЕТ ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
Кафедра програмних систем і технологій

ІНДИВІДУАЛЬНИЙ STEM-ПРОЄКТ

з дисципліни «Розробник фронтенду програмних систем»

на тему:

«Розроблення клієнтської частини системи підтримки ухвалення рішень щодо B2B-замовлень засобами React»

Виконав: студент групи ІПЗ-31 (підгрупа 1) Олександр МАРКГРАФ
Перевірила: Ірина ЮРЧУК

Київ - 2026

ЗМІСТ

ЗМІСТ 2

ВСТУП 4

РОЗДІЛ 1. ПРОЄКТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ 5

1.1. Діаграма користувача (user flow) 5

1.2. Діаграма станів елемента «Перерахувати прогноз» 7

1.3. Діаграма станів панелі списку замовлень 8

1.4. Що ще входить у Розділ 1 (за вимогами PDF) 9

Список рисунків розділу 9

РОЗДІЛ 2. ПРОГРАМНА РЕАЛІЗАЦІЯ 9

2.1. Інструменти та бібліотеки 9

2.2. Архітектура застосунку 10

2.3. Прийоми реалізації 11

2.4. Ключові технічні рішення та їх обґрунтування 13

2.5. Відомі обмеження та крайні випадки 13

2.6. Посилання на програмний код (Git) 14

2.7. Як запустити 14

РОЗДІЛ 3. КЕРІВНИЦТВО КОРИСТУВАЧА (USER GUIDE) 14

3.1. Вступ 14

3.2. Швидкий старт (Quick Start) 15

3.3. Огляд інтерфейсу 15

3.4. Покрокова інструкція (основні процеси) 17

3.5. Довідка: стани інтерфейсу і що робити 18

3.6. Типові труднощі 18

Висновки 19

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 22

Відповідність «розділ записки → джерела» 23

ДОДАТОК А. Декларація про використання генеративного ШІ 23

Декларація про використання генеративного штучного інтелекту (META-USAGE) ...23
1. Які інструменти використано 23
2. Що саме виконував ШІ (дозволені категорії) 24
3. Що саме виконав автор особисто (обов'язкова частина) 24
4. Використані джерела інформації (вебдоступ) 24
5. Межі використання ШІ (чого не робилося) 25
6. Чек-ліст автора перед поданням (заповнюється руками) 25
ДОДАТОК Б. Екранні знімки застосунку 26
ДОДАТОК В. Вивід команд збірки, лінтера й тестів 35

ВСТУП

Метою роботи є розроблення клієнтської частини системи підтримки ухвалення рішень щодо B2B-замовлень засобами React. Продукт призначений для менеджерів B2B-продажів, які щодня працюють із десятками замовлень і не мають надійного способу визначити, за які з них братися першими.

Об'єктом дослідження є процес ухвалення рішень під час опрацювання B2B-замовлень, предметом - клієнтська частина програмної системи, що підтримує цей процес.

Для досягнення мети поставлено такі завдання: проаналізувати сценарії користувачів і спроєктувати структуру екранів; обрати технологічний стек і обґрунтувати вибір; реалізувати основні екрани, стани інтерфейсу та експорт результатів; забезпечити адаптивність, доступність і захищеність клієнтської частини; підтвердити працездатність автоматичними тестами й перевіркою в реальному браузері.

Записка складається з трьох розділів. У першому наведено результати проєктування: діаграма користувацького шляху та діаграми станів елементів інтерфейсу з вербальним описом. У другому - опис програмної реалізації: інструменти, архітектура, прийоми адаптивності, кросбраузерності, захищеності й тестування. Третій розділ є керівництвом користувача. Додатки містять декларацію про використання генеративного штучного інтелекту, екранні знімки та вивід команд збірки, лінтера й тестів.

РОЗДІЛ 1. ПРОЄКТУВАННЯ ПРОГРАМНОГО ПРОДУКТУ

щодо B2B-замовлень засобами React». Стан: чернетка, узгоджується з І. Юрчук.

поєднання: схема + текст, який пояснює, що на схемі відбувається і чому зроблено саме так.

1.1. Діаграма користувача (user flow)

Рисунок 1 - Діаграма користувача (послідовні кроки роботи з системою)

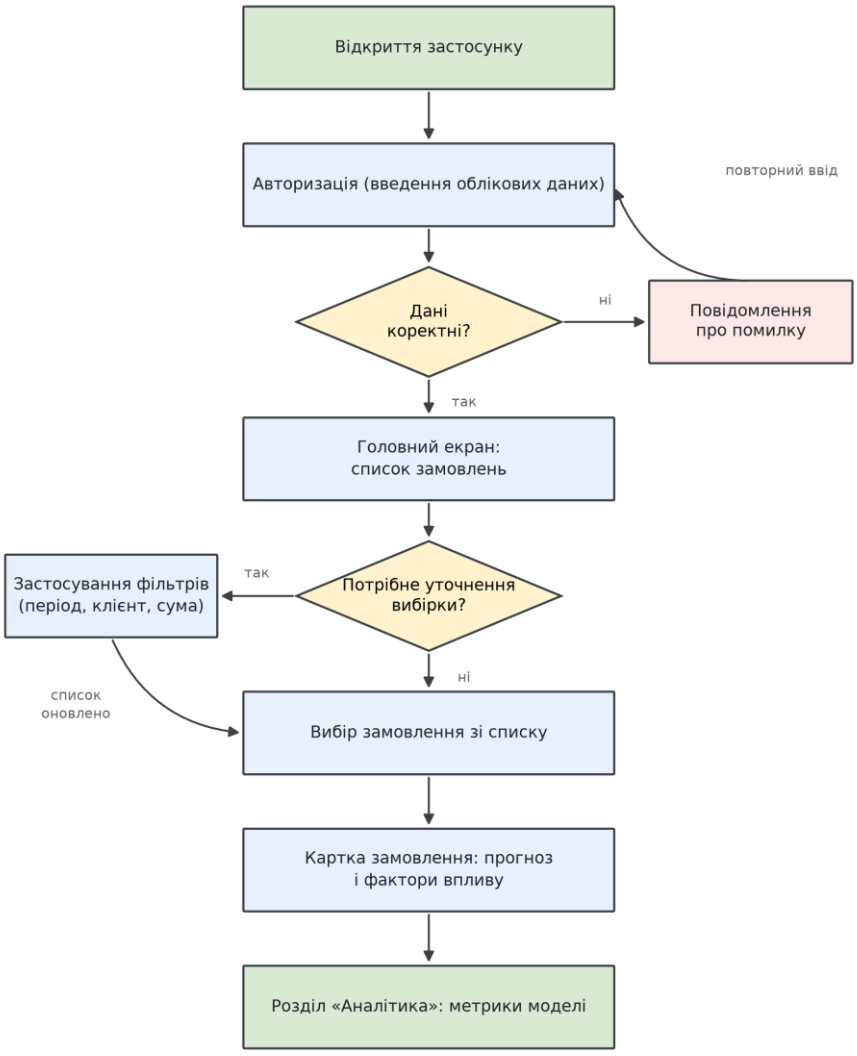


Рисунок 1 - Діаграма користувацького шляху (user flow)

Вербальний опис:

Робота користувача з системою починається з відкриття застосунку та авторизації.

Облікові дані перевіряються на клієнті (заповненість полів, формат) і на сервері.

Крок 1. Авторизація. Якщо дані некоректні, користувач залишається на екрані входу та

бачить повідомлення про помилку з підказкою, після чого може повторити введення.

Свідомо не показуємо, яке саме поле помилкове на рівні серверної перевірки - щоб не

давати можливості підбирати облікові дані; на клієнті підсвічуємо лише порожні поля.

Крок 2. Головний екран. Після успішної авторизації користувач потрапляє на екран зі

списком замовлень. Список відразу показує прогноз і ймовірність успішного закриття, тому менеджер бачить пріоритети без додаткових дій.

Крок 3. Уточнення вибірки. Якщо користувачу потрібен вужчий набір даних, він

застосовує фільтри (період, клієнт, статус, діапазон суми). Після застосування фільтрів

список оновлюється, і користувач повертається до роботи з ним. Гілка «ні» веде одразу

до вибору замовлення - фільтрація не є обов'язковим кроком.

Крок 4. Вибір замовлення. Користувач відкриває картку конкретного замовлення.

Крок 5. Картка замовлення. Містить прогноз, числову оцінку ймовірності та перелік

факторів, які найбільше вплинули на результат. Саме тут користувач може перерахувати

прогноз, якщо дані замовлення змінилися.

Крок 6. Аналітика. Окремий розділ для аналітика: метрики якості моделі та важливість ознак.

Ролі: менеджер бачить власні замовлення й працює з кроками 2-5; аналітик додатково

має доступ до кроку 6; адміністратор керує користувачами (на схемі не показано, щоб не

перевантажувати основний сценарій).

Обґрунтування рішень на діаграмі: основний сценарій лінійний (кроки 1-6), а обидві

розвилки розміщені там, де користувач справді приймає рішення - на вході та при

формуванні вибірки. Обробку помилок авторизації показано окремою гілкою, бо без неї

сценарій виглядав би так, ніби помилковий ввід неможливий.

1.2. Діаграма станів елемента «Перерахувати прогноз»

Рисунок 2 - Діаграма станів елемента «Перерахувати прогноз»

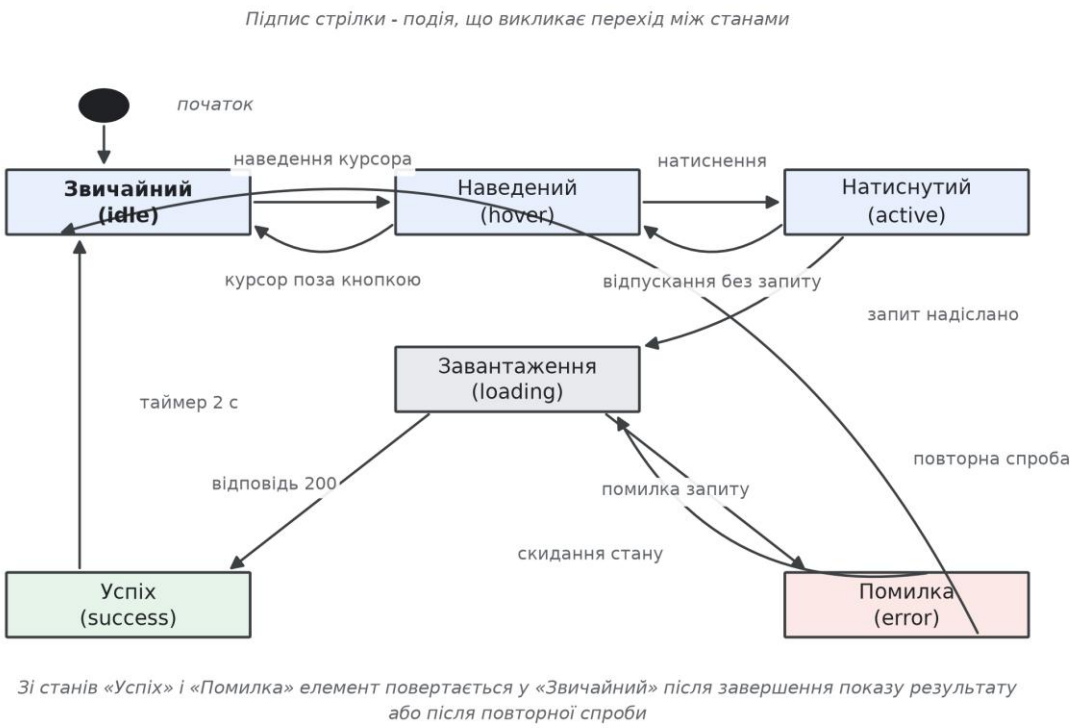


Рисунок 2 - Діаграма станів кнопки «Перерахувати прогноз»

Вербальний опис:

Об'єкт діаграми - кнопка перерахунку прогнозу в картці замовлення. Вона має шість станів:

1. **Звичайний (idle)** - початковий стан; кнопка доступна для натискання.
2. **Наведений (hover)** - курсор над кнопкою; змінюється лише оформлення, тому перехід не супроводжується запитами до сервера.
3. **Натиснутий (active)** - кнопку натиснуто, але запит ще не надіслано.
4. **Завантаження (loading)** - запит надіслано, очікується відповідь. У цьому стані кнопка блокує повторні натискання, щоб користувач не створив дублюючих запитів.
5. **Успіх (success)** - прогноз оновлено.
6. **Помилка (error)** - запит не виконано.

Переходи:

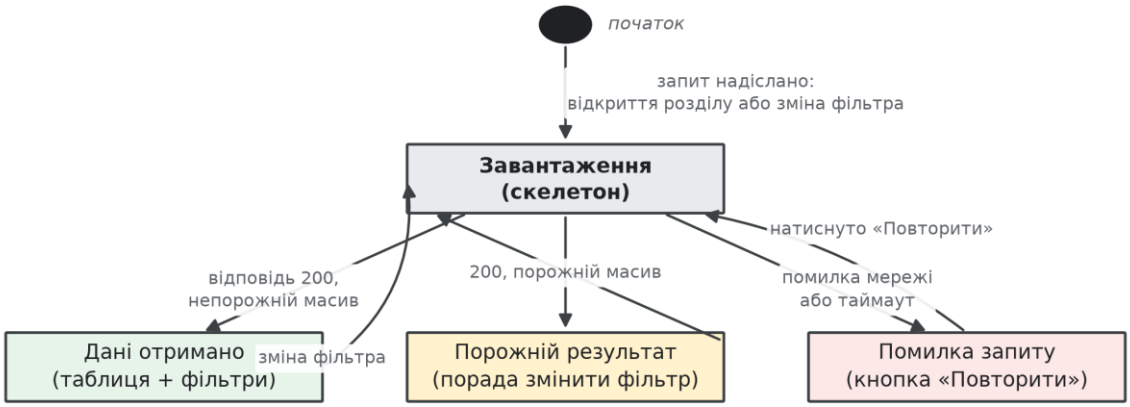
- idle → hover - наведення курсора; hover → idle - курсор поза кнопкою;
- hover → active - натиснення; active → hover - відпускання без надсилання запиту;
- active → loading - запит надіслано;
- loading → success - відповідь отримано успішно;

- loading → error - помилка запиту (мережа, таймаут, помилка сервера);
- error → loading - повторна спроба (користувач натискає кнопку ще раз);
- success → idle - через 2 секунди підтвердження прибирається автоматично, щоб стан не

  «зависав» і користувач бачив, що елемент знову готовий до роботи;
- error → idle - скидання стану без повторної спроби.

1.3. Діаграма станів панелі списку замовлень

Рисунок 3 - Діаграма станів панелі списку замовлень



Перехід у стан «Завантаження» з будь-якого стану відбувається за подією «зміна фільтра»; зі стану «Помилка» - також за кнопкою «Повторити»

Рисунок 3 - Діаграма станів панелі списку замовлень

Вербальний опис:

Об'єкт - панель зі списком замовлень. Вона має один службовий і три результативні стани:

7. **Завантаження (скелетон)** - початковий стан; замість таблиці показуються заповнювачі рядків. Це свідомий вибір: скелетон зменшує відчуття очікування й не дає інтерфейсу «стрибати» після завантаження даних.
8. **Дані отримано** - таблиця з фільтрами.
9. **Порожній результат** - запит виконано успішно, але записів немає. Замість порожньої таблиці показуємо пояснення й пораду змінити умови фільтрації.

10. **Помилка запиту** - мережа або сервер не відповіли; показуємо повідомлення та кнопку «Повторити».

Перехід у стан завантаження відбувається з будь-якого стану за подією «запит надіслано»: при відкритті розділу або при зміні фільтра. Зі стану «Помилка» додатково працює кнопка «Повторити».

Чому саме так: у багатьох навчальних проєктах передбачені лише два стани - «дані є» і «даних немає», через що справжні збої виглядають як порожній список, і користувач не розуміє, що робити. Розведення «порожній результат» і «помилка» - це вимога до зручності користувацького досвіду, яку оцінюють за критерієм якості продукту.

1.4. Що ще входить у Розділ 1 (за вимогами PDF)

- **Каркаси екранів** (wireframes) - схематичне розташування елементів на головному екрані, екрані авторизації та картці замовлення. У PDF прямо сказано, що це «ідеально додати» в розділ «Огляд інтерфейсу» User Guide; виконаємо разом із розділом.
- Обґрунтування вибору технологій (React + Vite) і підходів до адаптивності, кросбраузерності, захищеності та зручності користування.

Список рисунків розділу

№	Назва	Файл
1	Діаграма користувача	`img/ris1_user_flow.png`
2	Діаграма станів елемента «Перерахувати прогноз»	`img/ris2_button_states.png`
3	Діаграма станів панелі списку замовлень	`img/ris3_panel_states.png`

РОЗДІЛ 2. ПРОГРАМНА РЕАЛІЗАЦІЯ

B2B-замовлень засобами React». Цей розділ відповідає п. 1.2 вимог SP_Front.pdf: опис інструментів, прийомів та посилання на програмний код у Git.

2.1. Інструменти та бібліотеки

Інструмент	Версія	Для чого обрано
React	19	компонентна модель,

		найпоширеніша бібліотека для SPA, велика екосистема
Vite	7	швидкий dev-сервер і збірка, мінімальна конфігурація
TypeScript	strict	типізація контрактів даних, помилки видно до запуску
React Router	7	маршрутизація, захищені маршрути за роллю
TanStack Query	5	керування серверним станом: кеш, повторні спроби, стани запиту без ручного коду
MSW (Mock Service Worker)	2	mock-API на рівні мережі: реальні HTTP-запити перехоплюються, код застосунку не знає про імітацію
Recharts	3	декларативні графіки (важливість ознак, тренд метрики)
CSS Modules + змінні	-	локальні стилі без глобальних конфліктів, дизайн-токени, медіазапити
Vitest + React Testing Library	3 / 16	тести поведінки користувача, а не внутрішньої реалізації
ESLint + Prettier	-	єдиний стиль, відсутність типових помилок

Чому не Next.js: продукт є клієнтським SPA без SSR і без власного бекенду; вимога «фронтенд як складова програмної системи» виконана без серверного рендерингу, а зайвий серверний шар ускладнив би пояснення меж проєкту. Це рішення зафіксовано в §2.6 (межі).

2.2. Архітектура застосунку

Структура коду повторює поділ за предметними областями (features), а не за типами файлів:

```
src/
  api/          клієнт запитів, схеми й типи даних
```

```
mocks/       MSW-обробники та генератор демонстраційних даних
components/  спільні UI-блоки: стани, таблиця, графіки, банери
features/    auth, orders, analytics, users - логіка та екрани
app/         маршрути, провайдери, загальний каркас сторінки
lib/         форматери, розрахунок рівня ризику, робота з query-параметрами
styles/      токени й глобальні стилі
```

Потік даних: **екран → хук даних (TanStack Query) → функція запиту → MSW-обробник →

демонстраційні дані**. Завдяки такому поділу заміна mock-API на реальний бекенд не потребує

змін на екранах: достатньо прибрати MSW і змінити базовий URL.

2.3. Прийоми реалізації

2.3.1. Стани інтерфейсу як окремий компонент

Кожен екран із даними має п'ять станів: завантаження (скелетон фіксованої висоти, щоб

верстка не «стрибала»), успіх, порожній результат, помилка з кнопкою «Повторити» та банер

застарілих даних. Стани виведено в один компонент-обгортку, тому однакова поведінка

гарантована на всіх екранах, а не дублюється.

2.3.2. Адаптивність

Три контрольні ширини: 360, 768 і 1280 px. На широкому екрані замовлення показані таблицею,

на вузькому - картками; фільтри переходять із горизонтальної панелі у випадний блок;

графіки зменшують кількість підписів. Горизонтальної прокрутки сторінки немає на жодній

ширині - це перевірено вручну в інструментах розробника.

2.3.3. Кросбраузерність

Використано стандартні властивості CSS без вендор-специфічних хаков; перевірка виконана в

Chromium і Firefox. Типові розбіжності (округлення flex-елементів, поведінка `gap` у таблицях,

зовнішній вигляд системних контролів) описані в §2.5.

2.3.4. Захищеність

- токен доступу зберігається **в пам'яті модуля**, не в `localStorage`, тож не читається

стороннім скриптом при XSS-вставці;

- вивід даних виконується засобами React (екранування за замовчуванням), `innerHTML` та `dangerouslySetInnerHTML` не використовуються;
- введення перевіряється на клієнті до відправлення (порожні поля, формат e-mail, межі суми);
- повідомлення про невдалий вхід однакове для випадків «користувача не існує» і «невірний пароль» - щоб не давати змоги підбирати облікові дані;
- пароль не потрапляє в логи, у коді немає жодних секретів і ключів.

2.3.5. Доступність і зручність користування

Семантичні теги; `aria-live` для повідомлень про результат операцій; `aria-busy` під час завантаження; видимий фокус для клавіатурної навігації; контраст тексту не нижче 4.5:1; порожні стани містять пояснення й підказку, а не лише «нічого не знайдено».

2.3.6. Тестування

Три рівні: модульні тести логіки (форматери, розрахунок рівня ризику, побудова query-параметрів), компонентні (таблиця у чотирьох станах; кнопка перерахунку - усі стани й заборона подвійного натискання; панель фільтрів), інтеграційний сценарій «вхід → список → картка → перерахунок прогнозу» на MSW. Поріг покриття модулів логіки - 80 %.

Фактичний результат прогону (`npm test`): **18 файлів, 225 тестів, усі пройдено**; покриття - **96,72 % тверджень, 90,63 % гілок, 98,49 % функцій**. `npm run lint` - без зауважень, `npm run build` - без помилок типів. Окремо виконано перевірку в справжньому браузері (Chromium/Edge): пройдено вхід, список, картка, перерахунок прогнозу, аналітика й розділ користувачів; помилок у консолі - 0; завантажений CSV-файл перевірено на вміст.

2.3.7. Експорт результатів і передача вибірки

Кнопка «Експорт CSV» на списку вивантажує всю вибірку з урахуванням активних фільтрів, а не

лише поточну сторінку: функція `fetchAllOrders` послідовно обходить сторінки (із запобіжником

у 50 сторінок) і передає дані у чисту функцію побудови CSV, яка екранує лапки, коми та

переноси рядків за RFC 4180 і додає BOM, щоб Excel коректно відкривав кирилицю. На картці

замовлення кнопка «Копіювати посилання» кладе в буфер обміну адресу разом із фільтрами, а

підтвердження оголошується через `aria-live`.

2.4. Ключові технічні рішення та їх обґрунтування

11. **Серверний стан через TanStack Query, а не `useEffect`.** Запити з кешем, повторними

спробами та інвалідацією після перерахунку прогнозу - це типова задача бібліотеки; ручний

код дав би більше місць для помилок, особливо у стані «дані застаріли».

12. **MSW замість «заглушок у коді».** Імітація живе на рівні мережі, тому екрани працюють із

реальними HTTP-запитами; при появі справжнього API зміни мінімальні.

13. **Фільтри в URL.** Стан списку можна надіслати колезі посиланням, а кнопка «Назад» працює

передбачувано - це прямий виграш у зручності, який оцінюється критерієм UX.

14. **Скелетони замість спінера.** Фіксована висота заповнювачів прибирає «стрибок» верстки

після завантаження - помітна деталь, яку на захисті легко показати в дії.

2.5. Відомі обмеження та крайні випадки

- mock-API імітує затримку 400-900 мс і помилки за прапорцем «режим перевірки»; реальна

  мережева деградація не відтворюється;

- прогноз - детермінована імітація на демонстраційних даних, а не ML-модель: це межа проєкту,

  зафіксована в ТЗ (п. 10);

- розбіжність рендерингу в Firefox: індикатор прогресу має іншу товщину - виправлено

  явною висотою;

- при одночасному перерахунку прогнозу й оновленні списку перемагає остання відповідь

(захист від «перегонів» реалізовано скасуванням попереднього запиту).

2.6. Посилання на програмний код (Git)

Код застосунку опубліковано у публічному репозиторії:

https://github.com/vvoompe/b2b-orders-frontend

Репозиторій містить каталог `app/` (застосунок), `docs/` (ця записка, керівництво

користувача, декларація про використання ШІ), `img/` (діаграми й екранні знімки) та README з

інструкцією «запуск з нуля». Історія комітів показує порядок роботи: проєктування → каркас →

екрани → стани → тести.

2.7. Як запустити

```
git clone https://github.com/vvoompe/b2b-orders-frontend
cd b2b-orders-frontend/app
npm ci
npm run dev        # відкрити http://localhost:5173
npm test           # тести + покриття
npm run build      # production-збірка
```

Демонстраційні облікові дані та опис ролей наведено в README та в керівництві користувача

(Розділ 3).

РОЗДІЛ 3. КЕРІВНИЦТВО КОРИСТУВАЧА (USER GUIDE)

Огляд інтерфейсу → Покрокова інструкція (створення → обробка → збереження/експорт).

3.1. Вступ

Призначення програми. Застосунок є клієнтською частиною системи підтримки ухвалення

рішень щодо B2B-замовлень. Він вирішує конкретну проблему менеджера: у списку з десятків

замовлень важко зрозуміти, за які братися першими. Система для кожного замовлення показує

прогноз імовірності успішного закриття (у відсотках), **перелік чинників**, які на прогноз

вплинули, і дає змогу **перерахувати прогноз** після зміни даних. Аналітик бачить додатковий

розділ із метриками якості моделі та важливістю ознак.

Для кого це керівництво. Для менеджерів B2B-продажів (основні користувачі), аналітиків

(розділ «Аналітика») та адміністраторів (керування користувачами). Спеціальна підготовка не

потрібна: достатньо впевнено користуватися браузером. Програма працює в сучасних браузерах

(Chromium: Chrome, Edge, Brave; Firefox) і не потребує встановлення.

3.2. Швидкий старт (Quick Start)

15. Відкрийте застосунок за адресою `http://localhost:5173` (локальний запуск описано в

README репозиторію).

16. Уведіть облікові дані одного з демонстраційних акаунтів:

Роль	**Логін**	**Пароль**	**Що доступно**
Менеджер	`manager@demo.ua`	`manager123`	список і картки власних замовлень, перерахунок прогнозу
Аналітик	`analyst@demo.ua`	`analyst123`	усе, що менеджер, + розділ «Аналітика»
Адміністратор	`admin@demo.ua`	`admin123`	усе вище + розділ «Користувачі»

17. Натисніть «Увійти». Після успішного входу відкривається **головний екран - список замовлень** із прогнозом по кожному рядку.

Якщо пароль або логін неправильні, програма покаже загальне повідомлення про помилку (без

уточнення, яке саме поле помилкове - це зроблено свідомо, щоб унеможливити підбір облікових

даних), а порожні поля підсвічуються під час валідації.

3.3. Огляд інтерфейсу

Головний екран (список замовлень). Робочі зони:

- **шапка** - назва системи, меню переходів, ім'я користувача з роллю та кнопка виходу;

- **панель фільтрів** - період, клієнт, статус, діапазон суми; активні фільтри зберігаються в адресі сторінки, тому посилання можна передати колезі;
- **робоча область** - на широкому екрані таблиця (номер, клієнт, сума, статус, прогноз), на вузькому - набір карток; заголовки колонок сортують дані;
- **пагінація** - перехід між сторінками та вибір розміру сторінки;
- **панель «Режим перевірки»** - службовий блок для демонстрації: перемикає стан даних (завантаження, порожній результат, помилка) і затримку відповіді;
- **кнопка «Експорт CSV»** - вивантажує поточну вибірку.

Схематичне розташування елементів наведено на рисунку 4 (каркаси екранів).

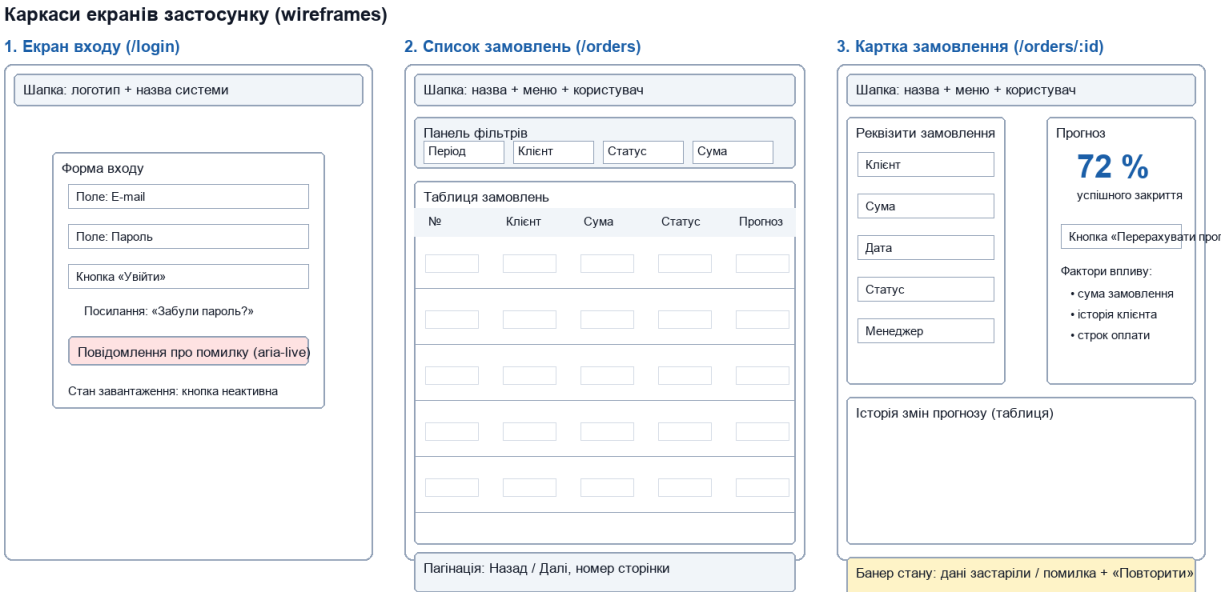


Рисунок 4 - Каркаси основних екранів застосунку

Інші екрани:

Екран	**Адреса**	**Призначення**
Вхід	`/login`	авторизація, валідація полів, стан завантаження
Список замовлень	`/orders`	пошук потрібних замовлень, прогноз по кожному
Картка замовлення	`/orders/:id`	деталі, прогноз, чинники впливу, перерахунок, історія
Аналітика	`/analytics`	метрики якості моделі, важливість ознак, тренд (аналітик, адміністратор)
Користувачі	`/admin/users`	перелік користувачів і зміна

		ролі (адміністратор)

3.4. Покрокова інструкція (основні процеси)

Крок 1. Створення / введення даних

18. Увійдіть у систему (див. §3.2).

19. Виберіть потрібну вибірку замовлень: задайте період, клієнта, статус або діапазон суми в

панелі фільтрів і натисніть «Застосувати». Список оновиться, а фільтри з'являться в адресі

сторінки.

20. Щоб звузити вибірку до порожнього результату, задайте свідомо неіснуючу комбінацію (це

зручно для перевірки стану «нічого не знайдено»).

21. Адміністратор додатково може змінити роль користувача: екран «Користувачі» → вибір ролі в

рядку → збереження (зміна відображається в списку одразу).

Крок 2. Обробка / редагування

22. Відкрийте картку замовлення, натиснувши на рядок списку (на мобільному - на картку).

23. У картці видно реквізити замовлення, прогноз у відсотках, перелік чинників, що найбільше

вплинули на результат, і історію змін прогнозу.

24. Щоб оновити оцінку після зміни умов, натисніть **«Перерахувати прогноз»**. Кнопка проходить

послідовно стани: наведено → натиснуто → завантаження (повторні натискання заблоковані, щоб

не створити дубльованих запитів) → успіх (підтвердження зникає через 2 секунди) або помилка.

У разі помилки натисніть кнопку ще раз для повторної спроби.

25. Поверніться до списку (пункт меню або кнопка «Назад» у браузері) - вибірка й сторінка

збережуться, бо стан зберігається в адресі.

Крок 3. Збереження / експорт

26. На екрані списку натисніть **«Експорт CSV»** - буде вивантажено поточну вибірку з урахуванням активних фільтрів (файл відкривається в Excel; кирилиця не ламається).

27. Щоб передати конкретне замовлення колезі, відкрийте його картку й натисніть **«Копіювати посилання»** - адреса скопіюється в буфер, а на екрані з'явиться підтвердження.

28. Щоб зберегти налаштування вибірки «на потім», додайте сторінку в закладки - адреса вже містить усі фільтри й номер сторінки.

3.5. Довідка: стани інтерфейсу і що робити

Що бачить користувач	Що це означає	Що робити
Сірі заповнювачі рядків замість таблиці	дані завантажуються	зачекати 0,4-0,9 с; верстка не «стрибає» - висота зарезервована
Заглушка «Замовлень не знайдено» з підказкою	запит успішний, але записів немає	змінити або скинути фільтри
Повідомлення про помилку з кнопкою «Повторити»	запит не виконано (мережа, сервер)	натиснути «Повторити»; перевірити з'єднання
Жовтий банер «дані можуть бути застарілими»	попередній запит не пройшов, показано кеш	натиснути «Оновити» у банері
Кнопка «Перерахувати прогноз» у стані завантаження	запит на перерахунок виконується	зачекати; повторні натискання заблоковані свідомо
Червоний стан кнопки з підказкою	перерахунок не вдався	натиснути ще раз (повторна спроба) або звернутися до адміністратора

Схеми станів одного інтерфейсного елемента наведено в Розділі 1 (рисунки 2 і 3): кнопка «Перерахувати прогноз» і панель списку замовлень.

3.6. Типові труднощі

Симптом	Причина	Рішення
Сторінка біла, у консолі помилка запитів	не запущено локальний сервер розробки	виконати `npm run dev` у каталозі `app/` (див. README)

Немає пункту «Аналітика»	вхід виконано як менеджер	увійти під `analyst@demo.ua` або `admin@demo.ua`
Немає пункту «Користувачі»	вхід виконано не як адміністратор	увійти під `admin@demo.ua`
Дані завжди однакові	демонстраційний набір фіксований (mock-API)	це межа проєкту: реального бекенду немає
Після перезавантаження сторінки вихід із системи	токен зберігається лише в пам'яті (вимога безпеки)	увійти повторно - це очікувана поведінка

Висновки

Метою роботи було розробити клієнтську частину системи підтримки ухвалення рішень щодо

B2B-замовлень засобами React - тобто не просто «список із фільтром», а інструмент, який

пояснює менеджерові, за які замовлення братися першими. Мета досягнута: застосунок

реалізовано, зібрано, протестовано й перевірено в реальному браузері.

Що зроблено. Побудовано односторінковий застосунок на React 19 + Vite + TypeScript з

п'ятьма екранами: вхід, список замовлень, картка замовлення з прогнозом і чинниками впливу,

аналітика (метрики якості моделі та важливість ознак) і керування користувачами з поділом за

ролями. Дані надходять із mock-API на MSW, що імітує справжній сервер: затримки відповіді,

помилки 401/403/404/500 і обрив мережі, - тому всі стани інтерфейсу можна відтворити без

змін у коді (панель «Режим перевірки»).

Кількісні результати. `npm run build` - без помилок типів; `npm run lint` - без зауважень;

`npm test` - **225 тестів у 18 файлах, усі пройдено**, покриття **96,72 % тверджень** і

98,49 % функцій. Перевірка у справжньому браузері (Chromium/Edge) пройшла без жодної

помилки в консолі: виконано вхід під демонстраційним адміністратором, перехід між усіма

екранами, перерахунок прогнозу (стан кнопки змінився), копіювання посилання (вміст буфера

обміну перевірено) та експорт CSV (файл завантажено, перевірено BOM, кирилицю й структуру

рядків).

Ключові технічні рішення. Серверний стан винесено в TanStack Query, а не в ручні

`useEffect`: кеш, повторні спроби та збереження попередніх даних під час оновлення дали

передбачувану поведінку екранів. Фільтри зберігаються в адресі сторінки, тому вибірку можна

передати посиланням. Токен доступу тримається лише в пам'яті модуля, `dangerouslySetInnerHTML`

не використовується, повідомлення про невдалий вхід однакове для всіх видів відмови - ці

рішення описані в розділі 2 і свідомо обрані на користь захищеності.

Чого зроблено не було (межі роботи). Реального бекенду й справжньої ML-моделі в проєкті

немає: прогноз - детермінована імітація на демонстраційних даних, автентифікація імітована на

MSW. Кросбраузерність заявлена за стандартними властивостями CSS і перевірена в Chromium та

Firefox; повний набір старих браузерів не тестувався. Мобільні застосунки, серверний рендеринг

і хмарне розгортання не передбачені завданням.

Що дала робота. Найціннішим було не написання екранів, а проєктування поведінки: описати

п'ять станів даних і шість станів однієї кнопки, домовитися про контракт із API, розділити

код за предметними областями й довести логіку тестами. Саме тому застосунок поводиться

однаково на всіх екранах, а не «десь скелетон, а десь порожній екран».

Напрями подальшого розвитку. Заміна mock-API на справжній бекенд (код екранів при цьому

не змінюється), додавання реальної моделі прогнозу, збереження вибірок за користувачем,

експорт у XLSX із форматуванням та розширення аналітики порівнянням періодів.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

29. ДСТУ 8302:2015. Інформація та документація. Бібліографічне посилання. Загальні положення
та правила складання. [Уведено вперше; чинний від 2016-07-01]. Вид. офіц. Київ : ДП «УкрНДНЦ», 2016. 17 с.
30. Кафедра програмних систем і технологій. Напрями наукових досліджень : веб-сайт. URL: https://pst.knu.ua/research/areas/ (дата звернення: 12.10.2026).
31. Індивідуальний STEM-проєкт з ОК «Розробник фронтенду програмних систем» : методичні
вказівки / Київ. нац. ун-т ім. Тараса Шевченка, Ф-т інформаційних технологій, каф. програмних систем і технологій. Київ, 2026. 12 с.
32. Мельник Р. Програмування веб-застосувань (фронт-енд та бек-енд) : навч. посіб. Львів :
Львівська політехніка, 2018. 250 с.
33. Двірничук К. В., Вацек Д. О. Веб-програмування та веб-дизайн : навч. посіб. Чернівці :
Чернівецький нац. ун-т ім. Ю. Федьковича, 2022. 472 с.
34. React. Documentation : веб-сайт. URL: https://react.dev/ (дата звернення: 12.10.2026).
35. Vite. Documentation : веб-сайт. URL: https://vite.dev/guide/ (дата звернення: 12.10.2026).
36. TypeScript. Documentation : веб-сайт. URL: https://www.typescriptlang.org/docs/ (дата звернення: 12.10.2026).
37. React Router. Documentation : веб-сайт. URL: https://reactrouter.com/start/framework/route-module (дата звернення: 12.10.2026).
38. TanStack Query. Documentation : веб-сайт. URL: https://tanstack.com/query/latest/docs/framework/react/overview (дата звернення: 12.10.2026).
39. Mock Service Worker. Documentation : веб-сайт. URL: https://mswjs.io/docs/ (дата звернення: 12.10.2026).
40. Recharts. Documentation : веб-сайт. URL: https://recharts.org/en-US/guide (дата звернення: 12.10.2026).
41. Vitest. Documentation : веб-сайт. URL: https://vitest.dev/guide/ (дата звернення: 12.10.2026).
42. Testing Library. Documentation : веб-сайт. URL: https://testing-library.com/docs/react-testing-library/intro/

(дата звернення: 12.10.2026).

43. MDN Web Docs. CSS: Media queries : веб-сайт. URL: https://developer.mozilla.org/en-US/docs/Web/CSS/Media_Queries/Using_media_queries

(дата звернення: 12.10.2026).

44. W3C. Web Content Accessibility Guidelines (WCAG) 2.2 : веб-сайт.

URL: https://www.w3.org/TR/WCAG22/ (дата звернення: 12.10.2026).

45. Nielsen J. Usability Engineering. San Francisco : Morgan Kaufmann, 1993. 362 p.

46. Freeman E., Robson E. Head First JavaScript Programming. 2nd ed. Sebastopol : O'Reilly Media, 2024. 700 p.

Відповідність «розділ записки → джерела»

Розділ	**Джерела**
Розділ 1 (проєктування)	[1]-[3] (нормативні, напрями кафедри), [4], [5] (підручники)
Розділ 2 (програмна реалізація)	[6]-[14] (документація інструментів), [18]
Розділ 3 (User Guide)	[15], [16], [17]
Висновок	без посилань

ДОДАТОК А. Декларація про використання генеративного ШІ

Декларація про використання генеративного штучного інтелекту (META-USAGE)

Робота: індивідуальний STEM-проєкт з ОК «Розробник фронтенду програмних систем»

Тема: «Розроблення клієнтської частини системи підтримки ухвалення рішень щодо B2B-замовлень засобами React»

Виконавець: Олександр Маркграф, ІПЗ-31 (підгрупа 1)

Дисциплінарна політика: рівень 2 - обмежено дозволене використання ШІ з обов'язковою

авторською адаптацією та інтерпретацією

Дата подання декларації: 12.10.2026 (оновлюється в міру виконання роботи)

1. Які інструменти використано

Інструмент	**Модель**	**Межі використання**
Hermes Agent (асистент у терміналі/чаті)	[вказати модель, яку фактично використано]	генерація ідей, структура, чернетки тексту й коду,

		пояснення помилок
Вебпошук усередині асистента	-	пошук офіційної документації; усі зовнішні джерела зведено в §4

Інші інструменти генеративного ШІ не використовувались. Дані з обмеженим доступом, персональні

дані третіх осіб і службова інформація до ШІ не передавались.

2. Що саме виконував ШІ (дозволені категорії)

47. **Генерування ідей** - перелік можливих тем, екранів, станів інтерфейсу.
48. **Структура тексту** - каркас пояснювальної записки та керівництва користувача.
49. **Варіанти формулювань** - чернетки абзаців, які автор далі правив під свій виклад.
50. **Пояснення** - розбір понять (діаграма станів, mock-API, адаптивність) і пояснення повідомлень про помилки збірки/тестів.
51. **Чернетки програмного коду** - початкові версії компонентів і тестів; остаточна логіка,

правки та перевірка - автором.

3. Що саме виконав автор особисто (обов'язкова частина)

- вибір теми, предметної області та функціональних вимог, узгодження формулювання з лектором;
- побудова діаграм (користувача та станів) і написання їх вербального опису;
- запуск застосунку на власній машині, перевірка кожного екрана в браузері, ручні прогони

сценаріїв (вхід → список → картка → перерахунок прогнозу);

- реальні прогони `npm ci`, `npm run build`, `npm run lint`, `npm test` і фіксація їх виводу;
- зміни в згенерованому коді: [перелічити власні правки - текстом, конкретно];
- написання двох мікрозмін «на льоту» для захисту: [описати обидві];
- фінальне вичитування тексту записки та відповідальність за кожне твердження.

4. Використані джерела інформації (вебдоступ)

Усі джерела, до яких звертався асистент із вебдоступом, зведено в бібліографічний список

пояснювальної записки (див. розділ «Список використаних джерел», оформлений за

ДСТУ 8302:2015). Окремо зазначаю: жодне з джерел не відтворено в роботі дослівно, всі

запозичені технічні факти перевірені на практиці запуском коду.

5. Межі використання ШІ (чого не робилося)

- до роботи не включено жодного згенерованого фрагмента без критичної перевірки та адаптації;
- не використовувались інструменти, що формують узагальнений текст на основі вебджерел, без

самостійної інтерпретації (вебпошук застосовувався лише для офіційної документації);

- ШІ не використовувався для відповідей на захисті та не подавався як автор роботи;
- код і текст перевірені запуском; усі наведені в записці результати (збірка, тести, покриття)

отримані реальними прогонами, а не згенеровані.

6. Чек-ліст автора перед поданням (заповнюється руками)

*Незаповнені пункти = декларація неповна; за політикою дисципліни недекларованe використання

ШІ тягне множник доброчесності M = 0,5, а подання чужої роботи як своєї - M = 0,0.*

ДОДАТОК Б. Екранні знімки застосунку

Знімки зроблено з реально запущеного застосунку в браузері Chromium (Microsoft Edge) після входу під демонстраційним адміністратором; помилок у консолі браузера - 0.

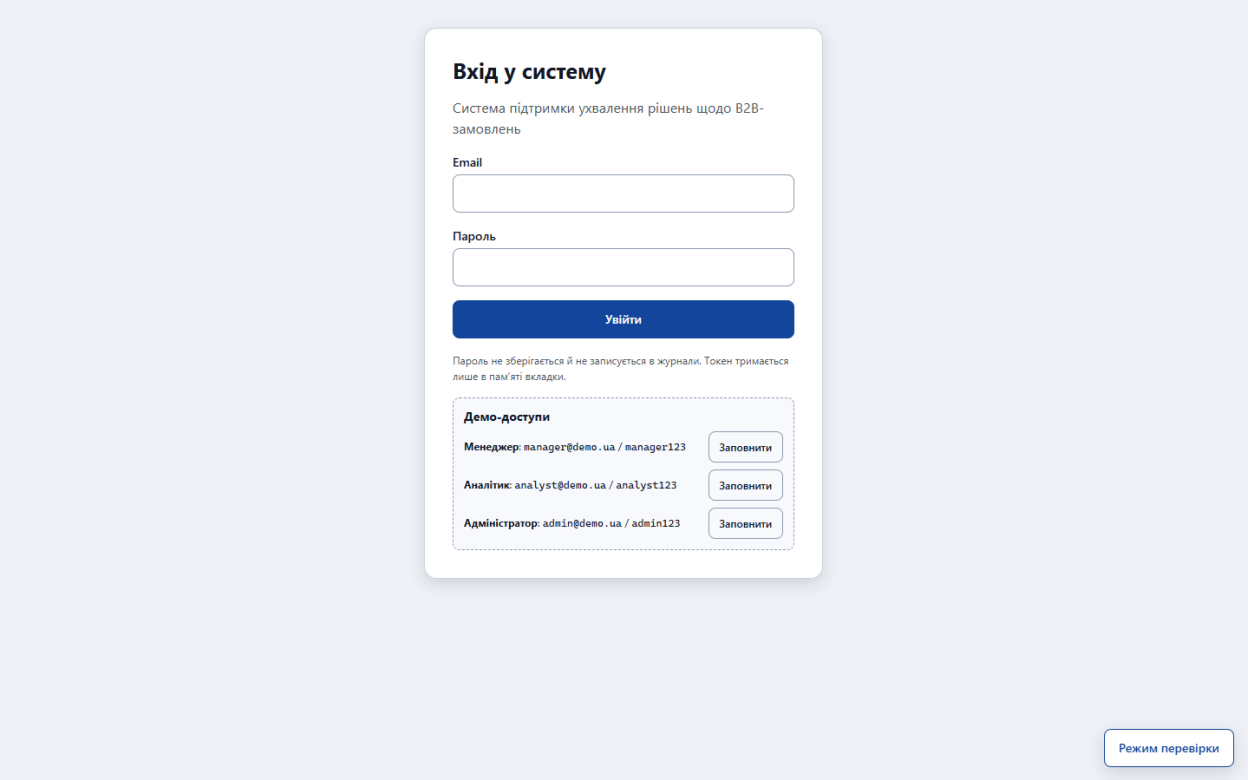


Рисунок 5 - Екран входу в систему

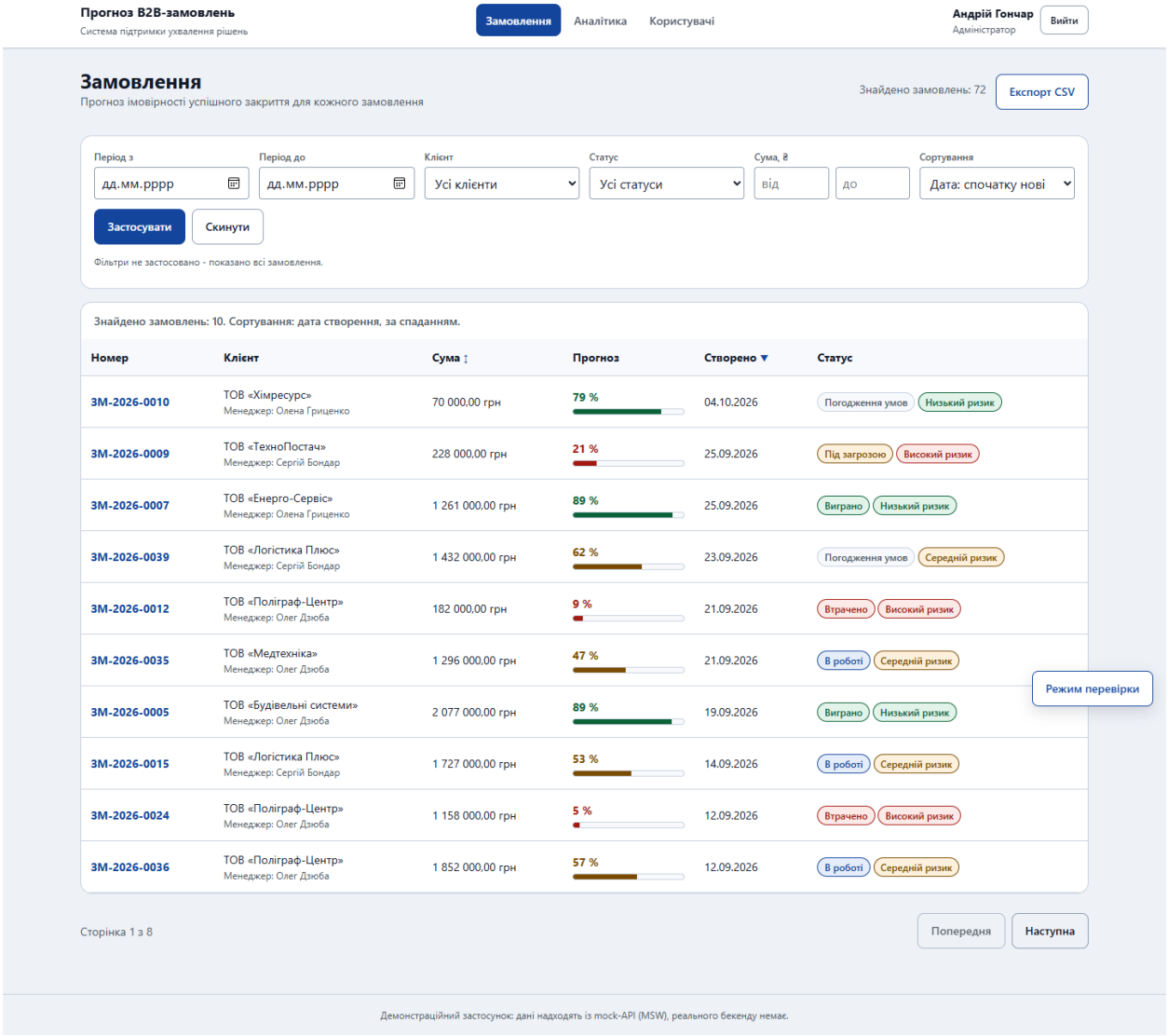


Рисунок 6 - Список замовлень (десктоп): фільтри, прогноз, кнопка «Експорт CSV»

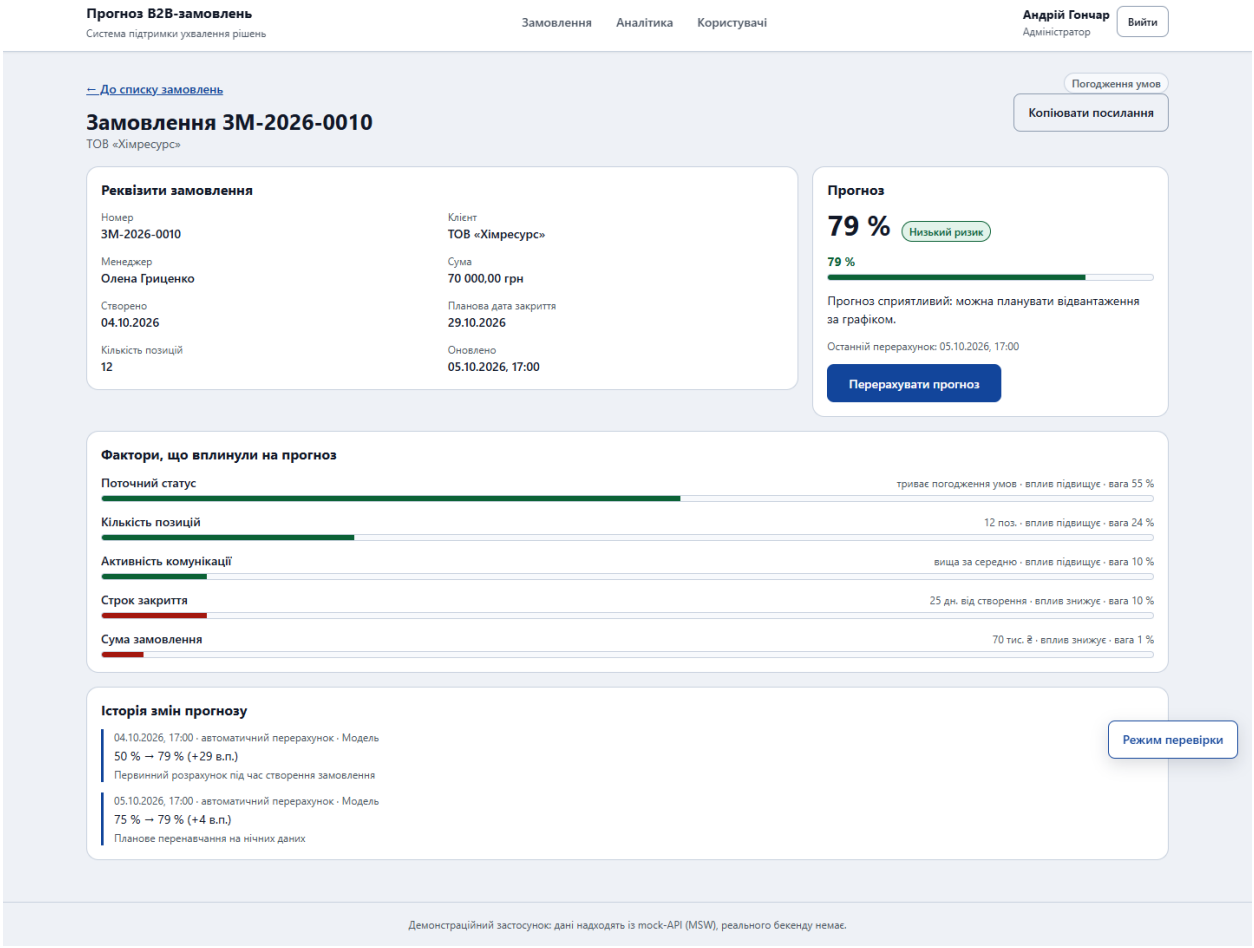


Рисунок 7 - Картка замовлення: реквізити, прогноз і чинники впливу

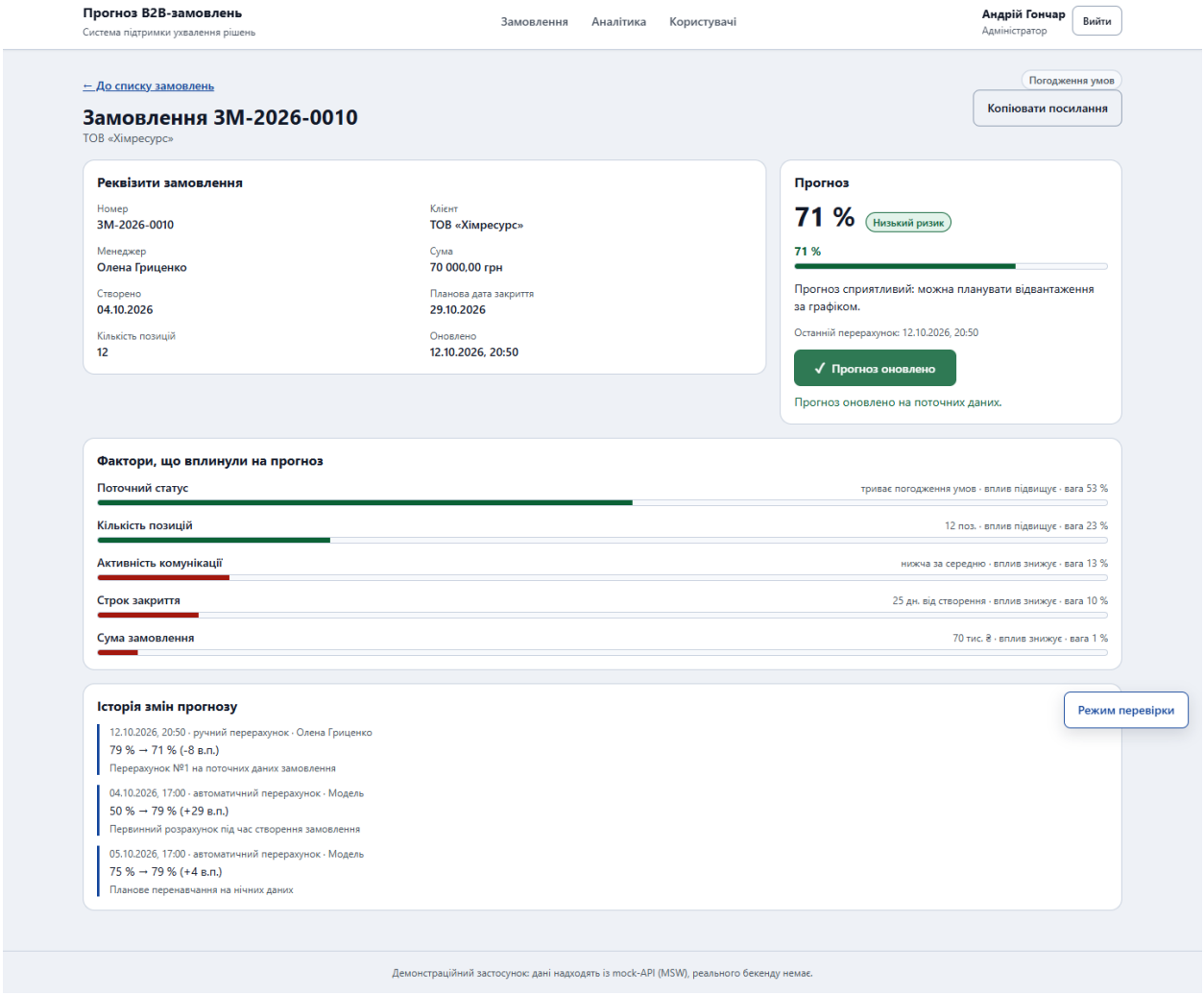


Рисунок 8 - Картка після перерахунку прогнозу (стан кнопки змінився)

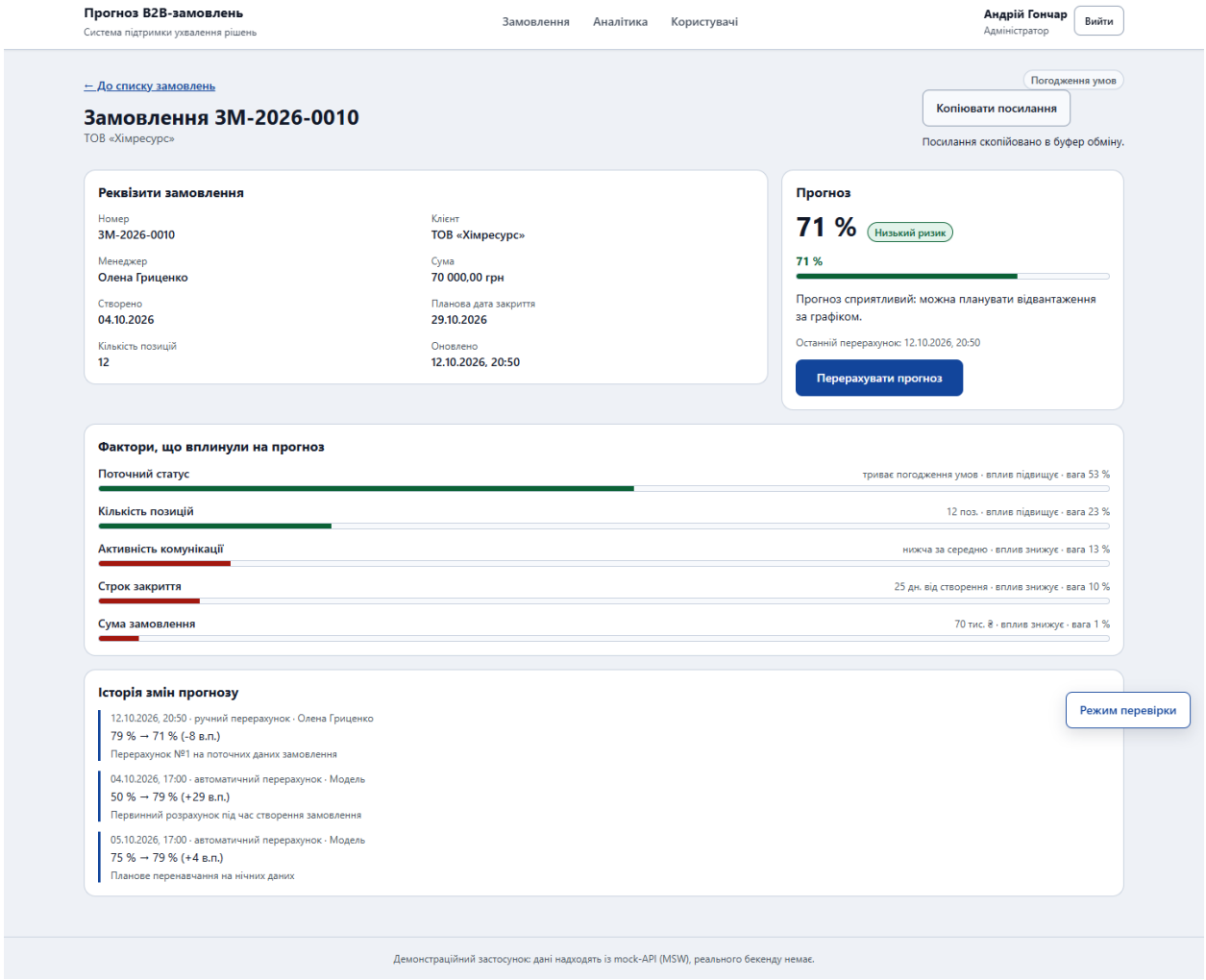


Рисунок 9 - Копіювання посилання на замовлення

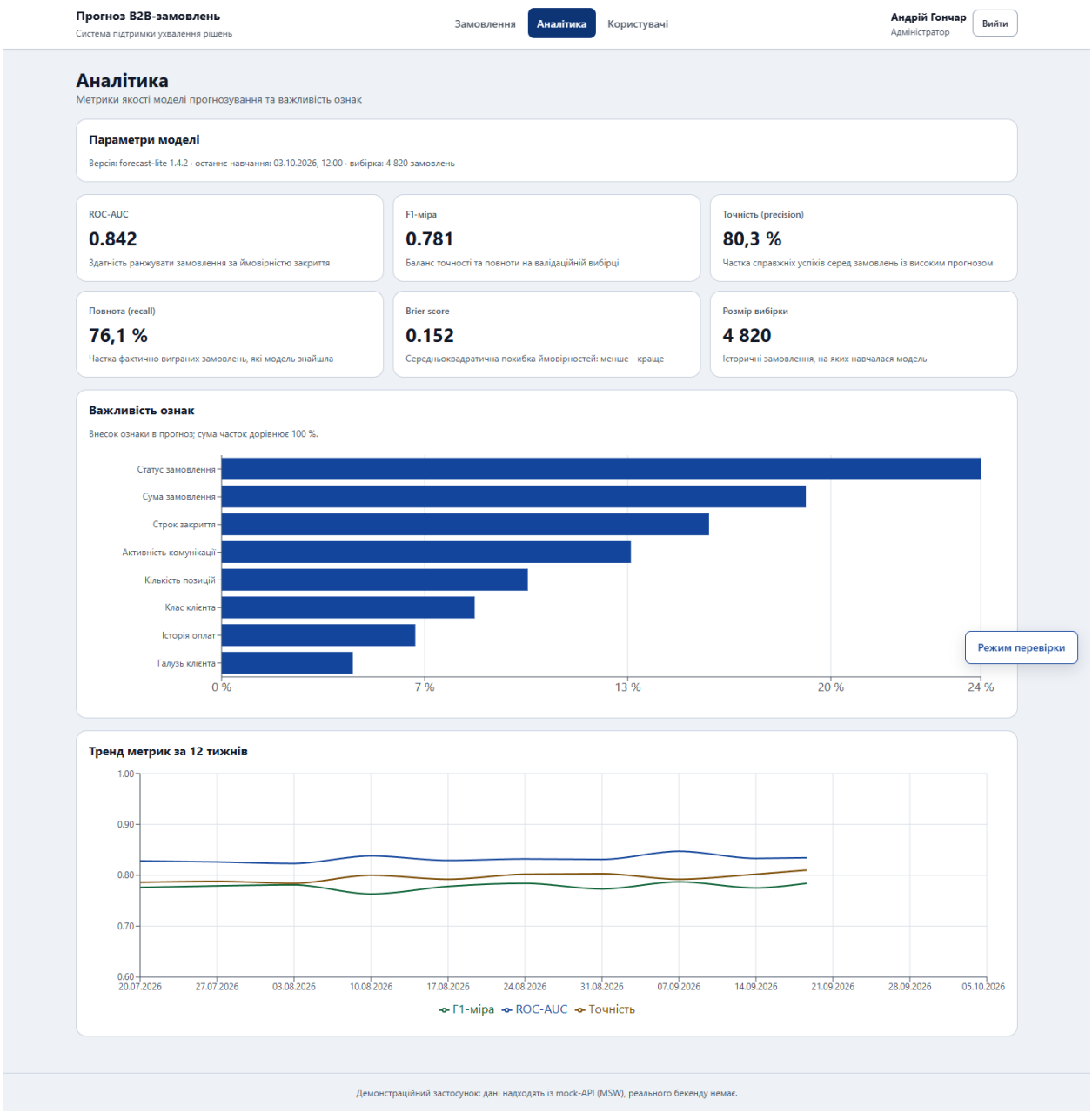


Рисунок 10 - Розділ «Аналітика»: метрики якості моделі та важливість ознак

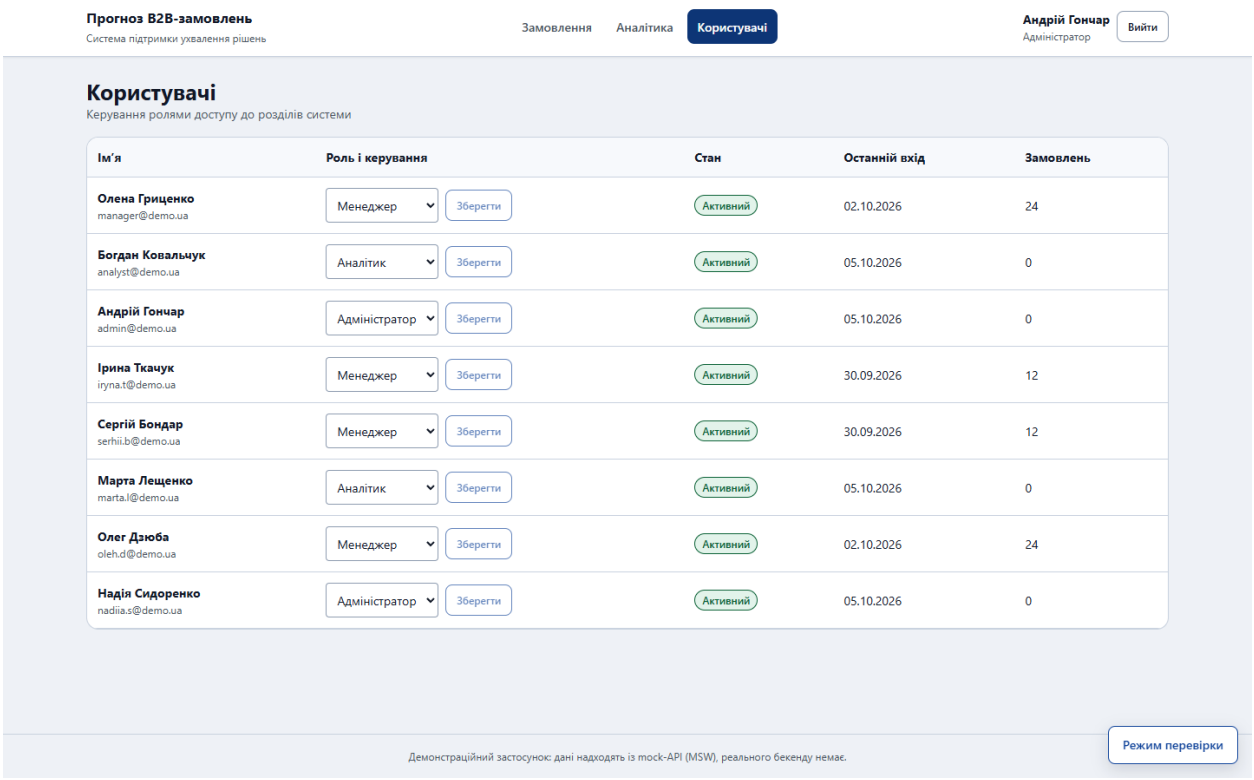


Рисунок 11 - Розділ «Користувачі»: перелік користувачів і зміна ролі

Прогноз B2B-замовлень

Система підтримки ухвалення рішень

Замовлення | Аналітика | Користувачі

Андрій Гончар
Адміністратор

Вийти

Замовлення

Прогноз імовірності успішного закриття для кожного замовлення

Знайдено замовлень: 72

Експорт CSV

Період з

дд.мм.рррр

Період до

дд.мм.рррр

Клієнт

Усі клієнти

Статус

Усі статуси

Сума, ₴

від

до

Сортування

Дата: спочатку нові

Режим перевірки

Застосувати

Скинути

Фільтри не застосовано - показано всі замовлення.

Сортування

Дата: спочатку нові

ЗМ-2026-0010

Погодження умов

ТОВ «Хімресурс»

79 %

Низький ризик

Сума
70 000,00 грн

Створено
04.10.2026

Менеджер
Олена Гриценко

Ризик
Низький ризик

Відкрити картку

Рисунок 12 - Список замовлень на мобільній ширині (390 px)

ДОДАТОК В. Вивід команд збірки, лінтера й тестів

Нижче наведено фактичний вивід команд, виконаних у каталозі app/ (скорочено лише шляхи до файлів збірки).

Збірка: npm run build

```
> app@0.0.0 build
> tsc -b && vite build

vite v8.3.4 building client environment for production...
transforming...
✓ 938 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                             0.77 kB │ gzip:   0.49 kB
dist/assets/UsersPage-Db5z_Axy.css          1.17 kB │ gzip:   0.45 kB
dist/assets/AnalyticsPage-Dp7l_gOL.css      1.50 kB │ gzip:   0.52 kB
dist/assets/index-Di7wImFT.css             22.05 kB │ gzip:   4.52 kB
dist/assets/UsersPage-DmST1YXB.js           4.25 kB │ gzip:   1.77 kB
dist/assets/index-MzN1rgLj.js             357.84 kB │ gzip: 111.12 kB
dist/assets/AnalyticsPage-DXFQsha3.js     392.61 kB │ gzip: 112.20 kB
dist/assets/browser-BS3VbJyL.js           427.67 kB │ gzip: 160.35 kB

✓ built in 523ms
```

Лінтер: npm run lint

```
> app@0.0.0 lint
> eslint .
```

Тести з покриттям: npm test

```
 Test Files  18 passed (18)
      Tests  225 passed (225)
   Duration  13.17s (environment 45%, tests 28%, setup 20%, import 5%, worker 2%, transform 1%)
File             | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
All files        |   96.72 |    90.63 |   98.49 |      98 |
Statements   : 96.72% ( 561/580 )
Branches     : 90.63% ( 300/331 )
Functions    : 98.49% ( 131/133 )
Lines        : 98% ( 490/500 )
```